WEBSITE HANDOVER GUIDE

# Nasser Al Ali Enterprises Website Go-Live Guide

A plain-English, step-by-step guide to taking this website from the zip file in your inbox to a fully working live site — every button, form and feature switched on.

**Target address: www.nasseralalienterprises.com**

Prepared for the IT team handling this launch · August 2026
No coding background required — every step is written out in full.

# Contents

# 1. What you were sent, in plain English

Before touching anything, here is what this project actually is and what is inside the zip file.

## What this website is

This is the full company website for Nasser Al Ali Enterprises, plus everything needed to run it. It is not just a set of web pages — it is a small self-contained system with four parts:

- **The public website** — the pages visitors see (Home, About, Services, Projects, Certifications & Awards, Careers, Contact, and more), in English and Arabic, with a language switch button.
- **An admin dashboard** — a private, password-protected control panel at the web address `/admin`. Staff use this to change text, swap photos, add awards, edit job listings, and more — without needing a developer.
- **A database** — a single file that stores every piece of text and every photo path shown on the site. When someone edits something in the admin dashboard, it updates this file.
- **An AI chat helper** — the blue chat bubble visitors can click to ask questions. It runs as a small separate program (a "Cloudflare Worker") so it can be turned on independently of the main website.

> **Why this matters**
> Because the website "remembers" things in a database and stores real uploaded photos, going live is not just "upload the files." A few extra steps make sure none of that real content gets left behind or wiped out. This guide walks through every one of them.

## What is inside the zip file

| Folder | What it is, in plain terms |
|---|---|
| `client` | The website itself — everything a visitor sees and clicks on. |
| `server` | The "engine" that runs behind the scenes: it serves the website, handles the admin dashboard, sends emails, and manages the database. |
| `server/data` | **The real, current content of the site** — one file (the database) holding all the live text and settings, including the recently corrected Chairman name and the recently added award photos. |
| `server/uploads` | The actual photo files currently used across the site (awards photos, certificates, client logos, and so on). |
| `workers` | The small separate program that powers the AI chat bubble. |
| `railway.json` | A ready-made instruction file that tells the hosting service how to build and start this site. You don't need to edit it. |

> **Check this first**
> Before you do anything else, confirm the `server/data` and `server/uploads` folders are actually present in the zip you received, and are not empty. These two folders hold the real, current photos and text (including the latest updates). If they are missing, ask whoever sent the zip to re-send it as a plain "compress to zip" of the whole project folder — not a GitHub download — because those two folders are deliberately left out of GitHub for privacy and size reasons.

## 2. Before you start

Create these accounts first. All of them are quick to set up, and most are free.

| Account | What it's for | Cost |
|---|---|---|
| GitHub (github.com) | Stores the website's code safely, and connects to the hosting service. | Free |
| Railway (railway.com) | Runs the website so it is live on the internet, 24 hours a day. | Usage-based, a few dollars a month for a small site |
| Cloudflare (cloudflare.com) | Runs the AI chat helper. | Free |
| Access to the domain's DNS panel | Wherever `nasseralalienterprises.com` was originally purchased (for example GoDaddy, Namecheap, or similar) — needed to point the domain at the live site. | Already owned |
| A Gmail account + "App Password" | So the website can send automatic emails (booking confirmations, job applications). | Free |
| Web3Forms (web3forms.com) Optional | A backup way for the contact form to deliver messages. | Free |
| Groq (console.groq.com) Optional | Powers the "auto-translate English to Arabic" button inside the admin dashboard. | Free tier available |

You will also need a computer with internet access, and about half a day set aside — most of the waiting is for automatic steps to finish, not typing.

## 3. Step 1 — Open the project on your own computer

Do this first, before putting anything online. It lets you see the real, working site on your own screen and catch problems early.

1. Unzip the file you received into an easy-to-find folder, for example a folder called `NAA` on your Desktop.
2. Install **Node.js** (the free program this website runs on). Go to `nodejs.org`, download the version marked **LTS** (version 20 or newer), and run the installer, clicking "Next" through the defaults.
3. Open a terminal (on Windows: search for "Command Prompt" or "PowerShell" in the Start menu; on Mac: search for "Terminal"). A terminal is simply a window where you type commands instead of clicking icons.
4. In the terminal, move into the project folder. Type `cd`, then a space, then drag the unzipped `NAA` folder into the terminal window (this fills in the path automatically), then press Enter.
5. Type the following and press Enter. This installs everything the project needs (it may take a few minutes):

   ```
   npm run install:all
   ```

6. Type the following and press Enter. This starts the website on your own computer:

   ```
   npm run dev
   ```

7. Open a web browser and go to `http://localhost:5173`. You should see the live website, including the About page with the Chairman's photo and corrected name, and the Awards & CSR gallery with the new photos showing first.
8. When you're done looking around, go back to the terminal and press `Ctrl + C` to stop it.

> **Tip**
>
> If step 7 shows the site correctly, the project itself is complete and working. Everything from here on is only about putting it on the internet under the real address — you are not fixing or building anything further.

## 4. Step 2 — Save the code on GitHub

GitHub is simply a safe, version-tracked place to store the website's code. The hosting service in the next step will read the code directly from here.

1. Go to `github.com` and create a free account if you don't already have one.

2. Click the **+** icon top-right, then **New repository**. Name it `naa-website`, set it to **Private** (important — this keeps the code from being publicly visible), and click **Create repository**.

3. Back in your terminal, make sure you're still inside the project folder, then type each of the following lines one at a time, pressing Enter after each:

```
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin PASTE_YOUR_REPOSITORY_LINK_HERE
git push -u origin main
```

4. Replace `PASTE_YOUR_REPOSITORY_LINK_HERE` with the link GitHub showed you after creating the repository (it looks like `https://github.com/yourname/naa-website.git`).

5. GitHub may ask you to sign in during this step — follow the on-screen prompts.

> **Why the data and uploads folders won't appear on GitHub**
>
> This project is deliberately set up so `server/data`, `server/uploads` and any `.env` files are *not* uploaded to GitHub. That's normal and correct — it keeps passwords and the live database out of code storage. Step 7 below explains how the real content gets onto the live site instead.

## 5. Step 3 — Create the hosting project (Railway)

Railway is the service that will keep this website running online, all day, every day.

1. Go to `railway.com` and sign up, choosing "Sign up with GitHub" so the two accounts are linked automatically.
2. Click **New Project**.
3. Choose **Deploy from GitHub repo**.
4. Find and select the `naa-website` repository you created in the previous step.
5. Railway will start building the project automatically. Don't worry if it fails or looks incomplete at this point — that's expected, because we haven't filled in the settings yet. Continue to the next steps.

## 6. Step 4 — Add permanent storage (the Volume)

By default, anything a hosting service stores can be wiped every time the site is updated. A "Volume" is a permanent storage box attached to the project so the database and photos are never lost.

1. Inside your Railway project screen, right-click on an empty part of the canvas and choose **New Volume** (or press `Ctrl/Cmd + K` and search for "Volume").
2. When asked which service to attach it to, choose your website service (it will usually be named after your GitHub repository).
3. When asked for a **mount path**, type exactly:

   ```
   /data
   ```

4. Save. The Volume is now created and permanently attached — nothing more to do here yet. We will put the real files into it in Step 7.

## 7. Step 5 — Fill in the settings (environment variables)

"Environment variables" is simply a technical name for a settings list — private values the website needs to run correctly (like where to save the database, and what email account to send from). None of this involves writing code.

1. In your Railway project, click on your website service, then click the **Variables** tab.

2. Add each row from the table below one at a time: click **New Variable**, type the name on the left exactly as shown, and the value on the right, then save.

| Name | Value to enter |
|---|---|
| `NODE_ENV` | `production` |
| `CLIENT_ORIGIN` | `https://www.nasseralalienterprises.com,https://nasseralalienterprises.com` |
| `DB_PATH` | `/data/app.db` |
| `UPLOAD_ROOT` | `/data/uploads` |
| `JWT_SECRET` | A long random password used only by the system itself (never typed by a person). See below for how to generate one. |
| `ADMIN_EMAIL` | The email address the main admin account should log in with, e.g. `owner@nasseralalienterprises.com` |
| `ADMIN_PASSWORD` | A temporary strong password — you will be forced to change it on first login. |
| `SMTP_HOST` | `smtp.gmail.com` |
| `SMTP_PORT` | `465` |
| `SMTP_USER` | The company Gmail address used to send emails |
| `SMTP_PASS` | The Gmail "App Password" — see below for how to generate one. |
| `MAIL_FROM_NAME` | `Nasser Al Ali Enterprises` |
| `NOTIFY_EMAIL` | `info@nasseralalienterprises.com` (where new booking/enquiry alerts are sent) |
| `GROQ_API_KEY` Optional | Only needed for the admin dashboard's auto-translate button. Leave blank to skip. |
| `VITE_CHAT_API_URL` | Leave blank for now — filled in during Step 9. |

### How to generate the JWT_SECRET value

In your terminal (the same one from Step 1), type this and press Enter, then copy the long string of letters and numbers it prints out:

```
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
```

### How to generate the Gmail App Password (SMTP_PASS)

1. Sign in to the Gmail account that will send emails for the site.
2. Go to `myaccount.google.com/security`.
3. Turn on **2-Step Verification** if it isn't already on (Google will guide you through this with a phone number).
4. Once that's on, search the same Security page for **App Passwords**, open it, and create a new one — name it "NAA Website".
5. Google will show a 16-character password. Copy it and use it as the `SMTP_PASS` value. This is different from your normal Gmail password.

> **Keep this private**
> Never share these values outside your team, and never paste them into GitHub or any public place. Railway's Variables tab is the only place they should live.

## 8. Step 6 — Go live on a Railway address

1. Back in your Railway project, click the service, then click **Deploy** (or it may redeploy automatically after you saved the variables in the previous step).
2. Wait for the status to turn green. This usually takes two to five minutes.
3. Click **Settings** → **Public Networking** and note the web address Railway generated automatically (it will look like `something.up.railway.app` ).
4. Open that address in your browser. At this point the site should load — but with the original starter photos and text, not your real content yet. That's expected — the next step fixes that.

## 9. Step 7 — Move the real content in

This is the step that brings across the real, current photos and text — including the newly added award photos and the corrected Chairman name — from your zip file onto the live site.

1. In your terminal, install Railway's command-line tool:

   ```
   npm install -g @railway/cli
   ```

2. Log in:

   ```
   railway login
   ```

   This opens a browser window — approve the login there, then return to the terminal.

3. Make sure your terminal is inside the project folder (the same one from Step 1), then connect it to your Railway project:

   ```
   railway link
   ```

   Follow the on-screen list to pick your project and service.

4. Upload the real database file:

   ```
   railway volume files upload server/data/app.db /app.db
   ```

5. Upload the real photos folder. Because this uploads many files, use the interactive file browser instead:

   ```
   railway volume browse /
   ```

   Inside the browser that opens, create a folder called `uploads`, then upload every file and sub-folder from your local `server/uploads` folder into it. Exit the browser when finished (usually the `q` key).

6. Back in the Railway dashboard, open the **Variables** tab again and double-check `DB_PATH` is exactly `/data/app.db` and `UPLOAD_ROOT` is exactly `/data/uploads` (matching the folder names you just uploaded into).

7. Click **Deploy** once more to restart the service so it picks up the new files.

8. Reload the Railway web address from Step 6. The site should now show the real content — the correct Chairman name and photo, and the Awards & CSR gallery with the newest photos first.

**Why not just click "Seed content" instead?**

This project includes a shortcut command ( `npm run server:seed:content` ) that fills the database with a starting set of example content. **Do not run it on the live site** — it would erase the real, current content (including the recent award photos and name correction) and replace it with only the original starter examples. Only use that command on a brand-new project that has never had real content loaded.

## 10. Step 8 — Log in to the admin dashboard

1. Go to your site's address followed by `/admin` — for example `https://your-railway-address.up.railway.app/admin`.
2. Log in with the `ADMIN_EMAIL` and `ADMIN_PASSWORD` values you set in Step 5.
3. You will be asked to set a new, permanent password immediately — choose a strong one and store it somewhere safe.
4. Take a moment to look through the left-hand menu — every section of the website (Services, Projects, Awards, Chairman, Careers and more) can be edited here, with photo upload built in.

> **Tip**
> If the login page says "invalid credentials", it usually means the real database wasn't uploaded correctly in Step 7 (so the site is still using a fresh, empty database with no admin account yet). Go back and confirm the upload, or as a fallback, use the terminal command `railway run npm run server:seed` to create a fresh admin account from your `ADMIN_EMAIL` / `ADMIN_PASSWORD` variables.

## 11. Step 9 — Switch on the AI chat helper

The chat bubble visitors see in the corner of the site is a separate small program. It needs its own quick setup.

1. Sign up for a free account at `cloudflare.com` if you don't have one.
2. Sign up for a free account at `console.groq.com` — this is the AI service that actually answers the chat questions. Once logged in, create an API key and copy it somewhere safe.
3. In your terminal, move into the chat helper's folder:

   ```
   cd workers
   ```

4. Log in to Cloudflare from the terminal:

   ```
   npx wrangler login
   ```

5. Save your Groq key into the chat helper (paste it when prompted):

   ```
   npx wrangler secret put AI_API_KEY
   ```

6. Publish the chat helper:

   ```
   npx wrangler deploy
   ```

7. This prints a web address, for example `https://naa-chat.yourname.workers.dev`. Copy it.
8. Go back to Railway → your service → **Variables**, and set `VITE_CHAT_API_URL` to the address you just copied.
9. Click **Deploy** again on Railway so the website rebuilds with the chat helper connected.
10. Open the live site and click the chat bubble to confirm it replies.

## 12. Step 10 — Connect the real domain

This is the step that makes the site appear at **www.nasseralalienterprises.com** instead of the temporary Railway address.

1. In Railway, open your service, go to **Settings** → **Public Networking**, and click **+ Custom Domain**.
2. Type `www.nasseralalienterprises.com` and confirm.
3. Railway will show you two values to copy: a **CNAME** record and a **TXT** record. Keep this screen open.
4. In a new browser tab, log in to wherever the domain `nasseralalienterprises.com` was originally purchased, and find the **DNS settings** or **DNS management** page for the domain.
5. Add a new **CNAME** record: Host/Name = `www`, Value/Target = exactly what Railway showed you.
6. Add a new **TXT** record with exactly the Host and Value Railway showed you. Both records are required — the domain will not activate with only one of them.
7. Save, then return to the Railway tab. It can take anywhere from a few minutes up to a few hours for a green checkmark to appear next to the domain — this is normal internet "propagation" delay, not an error.
8. Once verified, visit `https://www.nasseralalienterprises.com` directly in your browser to confirm the real site loads there.

**Optional — making the plain domain work too**

Some visitors may type `nasseralalienterprises.com` without the "www". Most domain providers have a "domain forwarding" or "redirect" option in the same DNS settings page — set it to redirect to `https://www.nasseralalienterprises.com`. If your provider doesn't support that, ask them or consider moving DNS management to Cloudflare (free), which handles this automatically.

## 13. Step 11 — Test everything

Go through this list on the real, live domain before telling anyone the site is finished. Tick each one off.

- [ ] The site loads at `https://www.nasseralalienterprises.com` with a padlock icon in the address bar
- [ ] The language switch button correctly swaps the whole site between English and Arabic
- [ ] The About page shows the Chairman's photo and the name "Nasser Ali J Z Alali"
- [ ] The Certifications & Awards page shows all award photos, with the newly added ones appearing first
- [ ] Every page in the top menu opens correctly (Home, About, Services, Projects, Certifications & Awards, Careers, Promotions, Contact)
- [ ] The consultation/contact form on the Contact page submits successfully, and a confirmation email arrives
- [ ] A test job application on the Careers page uploads a CV successfully
- [ ] The AI chat bubble opens and replies to a test question
- [ ] The WhatsApp and phone-call floating buttons open correctly on a mobile phone
- [ ] The cookie-consent banner's Accept and Reject buttons both work
- [ ] Logging in at `/admin` works, and a small text edit made there appears on the live site within a few seconds
- [ ] Uploading a new photo through the admin dashboard works and appears correctly sized on the site

## 14. Keeping the site running

### Making future changes

Day-to-day content changes (new text, new photos, new job listings) should be made through the `/admin` dashboard — no code changes needed. Actual code changes (a new page, a design change) should be made in the project folder, then sent to GitHub the same way as Step 2 — Railway will automatically rebuild and redeploy the site within a couple of minutes of receiving them.

### Backing up the database regularly

From the Railway dashboard, open a terminal into the service (or run locally with `railway run`) and run:

```
npm run server:backup
```

This saves a timestamped copy of the database and automatically keeps the most recent 30 backups. Set a calendar reminder to check this monthly, or ask your host to schedule it automatically.

### A reminder about the AI chat helper

The chat helper does not automatically learn about changes made in the admin dashboard. If services, prices, or contact details change, someone will need to manually update the file `workers/chat-worker.js` and repeat the `npx wrangler deploy` command from Step 9.

## 15. Plain-English glossary

| Term | What it means |
|---|---|
| Repository (repo) | A folder of code stored on GitHub, with a history of every change ever made to it. |
| Deploy / Deployment | The act of publishing the latest version of the code so it becomes the live website. |
| Domain / DNS | The domain is the web address ( `nasseralalienterprises.com` ). DNS is the phone-book system that tells the internet which server that address points to. |
| Environment variable | A private setting the website reads when it starts up — like a password or a folder location. |
| Volume | Permanent storage attached to the hosting service, so files aren't lost when the site is updated. |
| Database | A single organised file that stores all the site's text and photo references. |
| API key / Secret | A private code that lets one program securely talk to another (for example, the chat helper talking to the AI service). |
| Terminal / Command line | A text-based window for typing instructions to the computer, instead of clicking icons. |
| CNAME / TXT record | Two types of entries added to a domain's DNS settings to point it at the hosting service and prove ownership. |